

# Static Scheduling of Weight Programming for DNN Acceleration with Resource Constrained PIM

XIN GAO and HONGYUE WANG, Shandong University/Quan Cheng Laboratory, China  
YIYAN CHEN, YUHAO ZHANG, and ZHAOYAN SHEN, Shandong University, China  
LEI JU, Shandong University/Quan Cheng Laboratory, China

Most existing architectural studies on ReRAM-based processing-in-memory (PIM) DNN accelerators assume that all weights of the DNN can be mapped to the crossbar at once. However, these studies are over-idealized. ReRAM crossbar resources for calculation are limited because of technological limitations, so multiple weight mapping procedures are required during the inference process. In this article, we propose a static scheduling framework which generates the mapping between DNN weights and ReRAM cells with minimum runtime weight programming cost. We first build a ReRAM crossbar programming latency model by simultaneously considering the DNN weight patterns, ReRAM programming operations, and PIM architecture characteristics. Then, the model is used in the searching process to obtain an optimized weight-to-OU mapping table with minimum online programming latency. Finally, an OU scheduler is used to coordinate the activation sequences of OUs in the crossbars to perform the inference computation correctly. Evaluation results show the proposed framework significantly reduces the weight programming overhead and the overall inference latency for various DNN models with different input datasets.

CCS Concepts: • **Hardware** → *Emerging architectures*; • **Computer systems organization** → *Neural networks*;

Additional Key Words and Phrases: PIM, ReRAM, resource-constrained, DNN accelerator

## ACM Reference format:

Xin Gao, Hongyue Wang, Yiyang Chen, Yuhao Zhang, Zhaoyan Shen, and Lei Ju. 2024. Static Scheduling of Weight Programming for DNN Acceleration with Resource Constrained PIM. *ACM Trans. Embedd. Comput. Syst.* 23, 6, Article 89 (September 2024), 22 pages.  
<https://doi.org/10.1145/3615657>

## 1 INTRODUCTION

**Processing-in-memory (PIM)** -based accelerators can perform *in-situ* computation and eliminate the data movement between memory and processing elements [1, 4, 8, 11, 23]. It has been well recognized as a promising solution to bridge the ever-increasing performance gap between

X. Gao and H. Wang contributed equally to this research.

This work was supported by the Natural Science Foundation of China grant number 92064008, Taishan Scholars Program, and Qilu Young Scholar Program of Shandong University.

Authors' addresses: X. Gao, H. Wang, and L. Ju (corresponding author), Shandong University/Quan Cheng Laboratory, China; emails: {gao.xin, wanghongyue}@mail.sdu.edu.cn, julei@sdu.edu.cn; Y. Chen, Y. Zhang, and Z. Shen, Shandong University, China; emails: cheniyiyan@sdu.edu.cn, yu\_hao@mail.sdu.edu.cn, shenzhaoyan@sdu.edu.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](https://permissions.acm.org).

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

1539-9087/2024/09-ART89 \$15.00

<https://doi.org/10.1145/3615657>

computation and memory access latency for memory-intensive applications including **deep neural networks (DNNs)**. Among various PIM design choices, benefiting from the DRAM-like access performance and the support of in-place **matrix-vector multiplication (MVM)** operations, **Resistive RAM (ReRAM)** based on DNN PIM architectures has been widely explored.

PRIME [3] provides a novel micro-architecture and circuit design for the ReRAM-based DNN PIM accelerator. ISAAC [21] further proposes a pipelined PIM architecture where individual crossbars are dedicated to each DNN layer. However, these approaches are based on over-idealized assumptions that an entire ReRAM crossbar can be activated in one cycle. In practice, the accumulated current deviation of ReRAM cells would severely hurt model inference accuracy [2]. As a practical alternative, SRE [36] proposes a ReRAM architecture to only activate a small sub-region (a.k.a. operation unit, OU) of each crossbar in every computation cycle. RePIM [25] and PattPIM [42] further propose OU-based pruning and reusing methods to reduce ReRAM space requirements based on this architecture.

All the above-mentioned studies assume that the overall ReRAM capacity is large enough to store the entire DNN parameters, and the offline weight programming cost can be ignored. However, given the current technology scaling [14, 35, 37], inevitably, the ReRAM PIM chip capacity on an area-constrained embedded or edge devices (typically dozens of Mbits) is substantially smaller than the weight size of current DNN models (e.g., 548 MB for VGGNet [22]). Hence, it is impossible to deploy the entire weights of the network on ReRAM offline before the inference stage. Instead, multiple stages for online programming of partial weights are needed and the programming latency cannot be ignored. Latest ReRAM technology advances in **multi-level cell (MLC)** design substantially improve the ReRAM density [10]. Meanwhile, with the ever-increasing model complexity and weight size of DNNs, the deployment of a DNN inference engine with weight size larger than the overall PIM capacity will continue to be a common practice in resource-constrained application domains. A few recent works [16–18] consider the constrained PIM resource, and propose some scheduling schemes to improve the inference throughput of batched images. However, utilizing value similarity to reduce the weight programming overhead has not been discussed.

In this article, our goal is to design a DNN inference framework for practical ReRAM PIM devices with limited capacity. The framework optimizes the overall inference latency by reducing the online weight programming latency via statically scheduling the weight deployment. To achieve this goal, our design principle is to map the model weights to ReRAM cells with OU granularity and explore the weight value similarity to reduce the programming overheads. The following challenges need to be addressed: (1) The DNN weight to ReRAM OU mapping should be re-designed to seek the maximum bit-wise reuse benefits; (2) How to correctly activate OUs at runtime given the offline generated mapping.

To address the first challenge, we categorize the various writing characteristics of ReRAM crossbar discussed in the literature work, and conduct empirical studies to identify the potential impact on the DNN weight programming latency. In particular, we build a model for the crossbar programming latency considering three important factors including (i) bit-wise SET and RESET operations; (ii) programming latency variation between parallel executed crossbars; (iii) distance between OUs and voltage drivers of the crossbar. For the second challenge, in the offline phase, a scheduling table for each crossbar of every weight programming process is generated, and the activation sequences of the OUs for each inference computation stage are recorded. During the online inference stage, the corresponding OUs are activated by decoding the value in the scheduling table.

In summary, in this article, we propose a ReRAM-based PIM framework to reduce the overhead of the runtime weight programming by statically mapping DNN weights to OUs with minimum programming latency. The proposed weight-to-OU scheme is based on the

abstracted programming latency cost model of the underlying ReRAM device, therefore can be adopted to difference ReRAM device technologies including **single-level cell (SLC)** or **multi-level cell (MLC)**. Simulation results with AlexNet, GoogleNet, ResNet50, ResNet101, VGG16, Transformer [34], ERNIE [36], RoBERTa [37], BERT<sub>base</sub> [35], and Chinese BERT<sub>base</sub> models show that the proposed scheme achieves significant performance speedup. Furthermore, we also show that the multiple stages of online weight programming during DNN inference have limited impact on the ReRAM endurance. The contributions of this article are summarized as follows:

- We observe that it is impractical to deploy a whole DNN network on ReRAM-based accelerators with area constraints. Time-consuming weight programming is needed and the programming latency is subject to the DNN weight patterns, programming operations, and the locations of different ReRAM cells.
- We propose a ReRAM-based PIM framework including similarity-based mapping, distance-aware mapping and balanced mapping strategies to reduce the latency of weight programming. Scheduling tables for each crossbar are maintained to ensure the correctness of OU activation during the online inference stage.
- We evaluate the proposed framework with several standard DNN models and several **natural language processing (NLP)** models. Evaluation results show that our proposed static scheduling strategy achieves a significant speedup, up to 52.91% of the overall latency is reduced compared with the state-of-the-art.

The rest of this article is organized as follows: Section 2 introduces the background of ReRAM PIM and its programming latency analysis. Section 3 illustrates the motivating examples of the weight programming overhead for DNN inference on resource constrained PIM devices. Section 4 briefly introduces the overview of the proposed framework, where the detailed design of each module is elaborated in Section 5. Section 6 presents the evaluation results and discussions. Finally, Section 7 concludes this article.

## 2 BACKGROUND

### 2.1 OU-Based ReRAM PIM Accelerator for DNN

Figure 1 shows a typical ReRAM-based accelerator for DNN model [19, 21, 38]. The accelerator consists of multiple tiles, where each tile consists of multiple **processing elements (PE)**, and each PE has multiple ReRAM crossbar arrays. During the DNN inference stage, input feature map data (converted to input wordline voltages) are used to slide over the weights of DNN filters (pre-programmed to ReRAM cells as conductance) to perform MVMs in the convolution and fully connected layers.

Numerous ReRAM-based DNN accelerators assume that an entire crossbar array can be activated in a single cycle. However, it has been observed that the accumulated effect of per-cell current deviation on the inference accuracy is non-negligible [36]. Instead, each crossbar array needs to be operated at a smaller granularity (typically between  $8 \times 8$  and  $32 \times 32$ ), called *operation unit (OU)*, to ensure the accuracy of computation. As a result, only one OU on each crossbar array can be activated in each computation cycle. Similar to recent studies [12, 24–26, 42], we adopt the OU-based ReRAM PIM architecture in this work.

Figure 2 shows an example of the DNN weights mapping process to OU-based ReRAM crossbar arrays for a convolution layer. Assume the OU size is  $2 \times 3$ , and there are 2 filters of size  $2 \times 2 \times 2$ . Each filter weight is of 3-bits precision and is stored in three **single-level ReRAM cells (SLC)** across three bitlines. During implementation, the input vector is broken down and sequentially transmitted to wordlines over multiple clock cycles, while simultaneously decomposing and

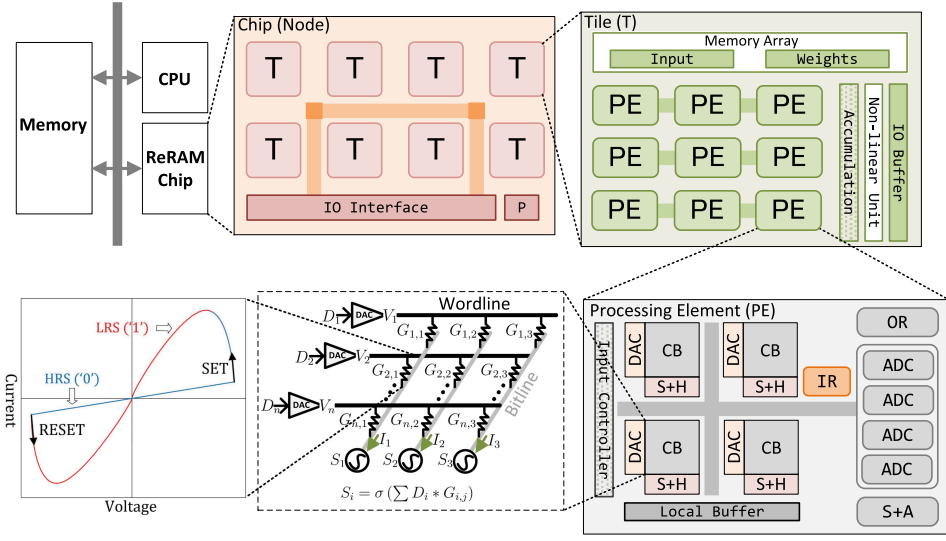


Fig. 1. A typical ReRAM-based DNN accelerator.

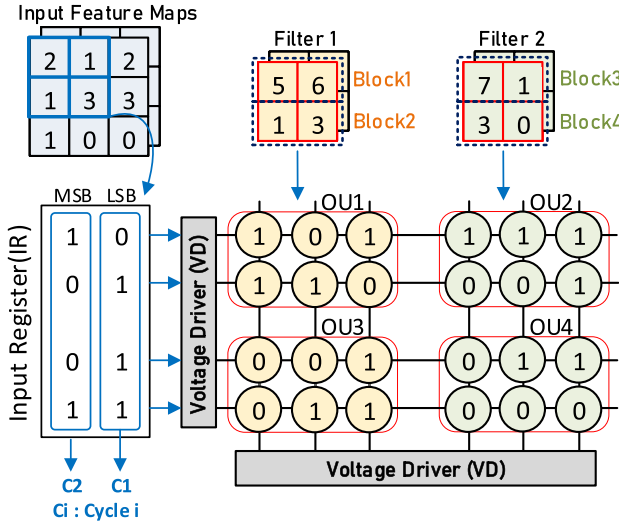


Fig. 2. Weight programming for OU-based ReRAM PIM.

mapping each data from the original filter weight onto different bitlines. For simplicity, the example only shows the mapping of the first channel for each of the two filters on the four OUs.

## 2.2 ReRAM Programming Latency

The programming latency of ReRAM cells is determined by multiple factors. First of all, it has been well recognized that a RESET operation takes a much longer time compared with a SET operation [28, 34, 39, 41]. The bottom left sub-figures in Figure 1 show the crossbar structure and the I-V characteristics of a typical bipolar ReRAM cell. A ReRAM cell with a **low resistance state (LRS)** and a **high resistance state (HRS)** can represent logic '1' and '0', respectively. Switching a cell

from HRS (logic ‘0’) to LRS (logic ‘1’) is a SET operation which needs a positive voltage, and the reverse process is a RESET operation which needs a negative voltage with a proper magnitude which needs significantly longer latency.

Furthermore, the programming procedure of a crossbar wordline also affects the latency. It is shown that the ReRAM effective voltage decays as the number of selected cells increases [29]. To prevent the potential write failures due to the effective voltage drops below the RESET threshold voltage, instead of programming the entire wordline of a crossbar array, a common strategy is to program the wordline at a smaller granularity of 4- or 8-bit segments [29, 30, 32, 34, 40]. For example, assume a  $512 \times 512$  crossbar array with 1024 OUs of size  $16 \times 16$ , the entire crossbar has to be programmed at the granularity of 8-bit segments sequentially, leading to  $1024 \times 16 \times (16/8) = 32,768$  programming times. Therefore, the overall programming latency of a PIM crossbar array is not negligible.

Finally, it has been observed that the distance between ReRAM cells and voltage drivers has significant impact on the programming latency [29]. The resistance switch of a ReRAM cell depends on the time integral of current. When the delivery path is longer, the equivalent line resistance in the circuit increases, resulting in higher voltage drops. This, in turn, weakens the current flowing through the cell. As a result, as depicted in Figure 2, the RESET latency of the cell in OU2 is significantly longer compared to that of OU3. In practice, for a  $512 \times 512$  crossbar array, a latency gap of up to 70X can be observed for RESET operations on cells with different distances to the voltage drivers [29].

### 3 MOTIVATION

Currently, most ReRAM-based DNN accelerators assume the entire weights of a DNN model can be pre-loaded into the PIM device without any runtime programming overhead. However, while the technology scaling continues to deliver denser and faster ReRAM-based PIM design, it is still lagging behind the ever-increasing complexity of DNN models. For example, as presented, with  $9\text{mm}^2$  and  $21.82\text{mm}^2$  ReRAM die sizes, the capacity can be only 64 Kb and 158.8 Kb [14, 37], respectively. With the die area increase, the capacity could be improved to several megabytes. Table 1 shows the weights of several common DNN and NLP networks based on 32-bit floating point and quantized into 8-bit intergers. We observed that these common models have weights of tens of MB or even hundreds of MB. Even with 4- or 8-bit weight value quantization, it still requires multiple or even dozens of weight programming stages to complete the inference for a single image on a ReRAM-based PIM device with practical area constraints (e.g., with 16 Mb or 32 Mb capacity). Even for MLC ReRAM with higher density, multiple rounds of weight programming (with potentially larger MLC write cost) are still inevitable under the same area constraint.

In a recent study [18], Lu et al. propose a scheduling framework for weight programming in the context of area constrained ReRAM PIM for DNN acceleration. However, they only know how to maximize the reuse of partially loaded weights via scheduling of input feature maps across a batch of input images. On the other hand, we aim to optimize the programming latency for each programming cycle by constructing the weight-to-OU mapping with fine-grained modeling of bit-level programming latency. The two approaches (reuse loaded partial weights and reduce weight programming latency) are orthogonal and can be combined seamlessly.

To show how much weight re-programming latency affects the system performance, we choose several typical neural network models and map them on capacity-constrained ReRAM PIM architecture for inference. The first six models are convolutional neural networks, in which those marked with † are trained based on dataset Imagenet and those marked with ‡ are trained based on dataset CIFAR10. The last three models are for natural language processing. The evaluation results are obtained on a 32-Mbit ReRAM with  $512 \times 512$  crossbar array and  $16 \times 8$  OU. For each

Table 1. Parameters Size of Several Common Neural Networks

| Neural Network         | 32-bit Float | 8-bit Integers |
|------------------------|--------------|----------------|
| AlexNet <sup>†</sup>   | 233.08 MB    | 58.27 MB       |
| ResNet50 <sup>†</sup>  | 97.69 MB     | 24.42 MB       |
| VGG16 <sup>†</sup>     | 527.79 MB    | 131.94 MB      |
| AlexNet <sup>‡</sup>   | 136.77 MB    | 34.19 MB       |
| ResNet101 <sup>‡</sup> | 162.57 MB    | 40.64 MB       |
| VGG16 <sup>‡</sup>     | 56.41 MB     | 14.10 MB       |
| Transformer            | 187.57 MB    | 46.89 MB       |
| BERT                   | 417.64 MB    | 104.41 MB      |
| RoBERTa                | 390.29 MB    | 97.57 MB       |

<sup>†</sup>Neural Network trained base on ImageNet.

<sup>‡</sup>Neural Network trained base on CIFAR10.

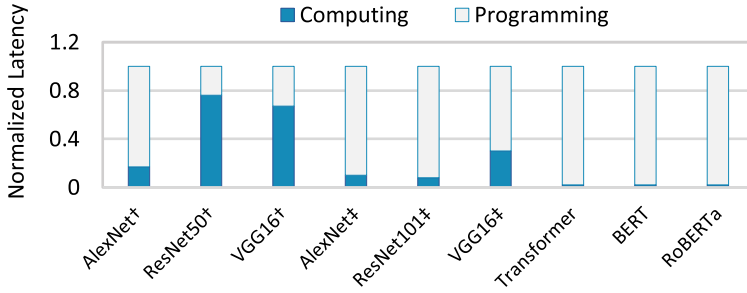


Fig. 3. DNN execution time on resource constrained PIM. <sup>†</sup> Neural Network trained base on ImageNet. <sup>‡</sup> Neural Network trained base on CIFAR10.

network, the input and weight are all quantized into 8 bits. Other experiment setup details will be presented in Section 6. The evaluation results are shown in Figure 3. As shown, we separate the execution time into weight programming time and computation time. The weight programming time is calculated by mapping OU-sized weight blocks to individual OUs with random initial values, and multiple weight programming stages are required subject to the total weight size and available PIM capacity. The weight-to-OU mapping follows a traditional inference-latency-centric manner, while the potential impacts on programming latency (as discussed in Section 2.2) are not considered.

The results show that the execution time breakdown of computation and programming of different neural networks present different distributions (e.g., 33% programming time for AlexNet base on ImageNet, 70% programming time for VGG16 base on CIFAR10, 97% programming time for BERT). For the models trained based on CIFAR10, since the input sizes are much smaller than that of ImageNet, the models have less convolution operations. Thus, the programming time for these models occupy higher percentage. For the model AlexNet, since its convolution kernel is larger than the other two models (i.e., ResNet50 and VGG16), it requires more programming time during the inference process. For the NLP models (i.e., Transformer, BERT, RoBERTa), the full connection layers are dominant, whose weights only need to be calculated one time after being programmed into ReRAM, thus, their execution time are mainly composed of programming process.

In summary, the programming time occupies a significant portion of the overall inference execution time for all the neural network models. In this article, we aim to optimize the weight programming process and propose a programming latency aware weight-to-OU mapping to

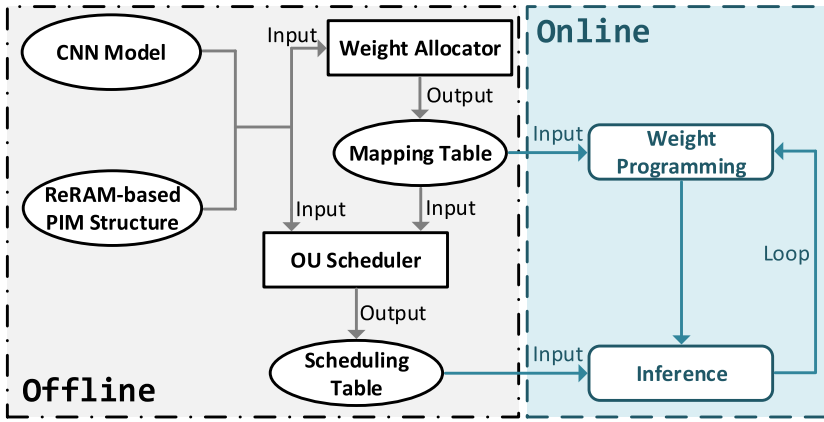


Fig. 4. Overview of the framework.

effectively improve the overall system performance of DNN inference on capacity-limited ReRAM PIM.

#### 4 FRAMEWORK OVERVIEW

With resource-constrained PIM device, the entire size of DNN model weights are much larger than the available PIM capacity. Hence, the DNN inference process has to be explicitly partitioned into multiple iterations. In each iteration, only partial weights can be programmed as resistances on the device. After one iteration, the device resistances need to be reprogrammed with weights belong to the next iteration, and the reprogramming processes are quite time-consuming as presented above.

In this section, we briefly introduce the proposed programming latency aware framework for DNN inference on resource constrained PIM devices. As shown in Figure 4, the proposed framework consists of an offline phase and an online phase. In principle, the offline phase is used to statically generate the weight-to-OU mappings with minimal reprogramming latencies between two inference iterations. With the weight-to-OU mapping, the offline phase also needs to maintain a OU scheduling scheme to coordinate the computation sequence of OUs in one iteration. The online phase utilizes the mapping and scheduling scheme to perform the weight programming and DNN inference at runtime.

##### 4.1 Offline Phase

The ultimate goal of the offline phase is to mitigate the overall ReRAM written cost of weight programming stages during the online phase. Since we adopt the OU-based ReRAM PIM architecture for DNN inference as discussed in Section 2.1, the task of the “Weight Allocator” module is to generate a weight-to-OU “Mapping Table”, which assigns each OU-sized weight block to a physical OU at a specific inference iteration. In accordance with the discussion in Section 2.2, the cost of writing an OU-sized weight block to a physical OU depends on various factors including the existing data stored in the OU as for the previous inference iteration (which determines the SET/RESET operations required), as well as the relative position of the OU within a crossbar array. Furthermore, during the weight programming stages at runtime, different crossbar arrays can be programmed in parallel, while weights blocks within each crossbar array are written in sequential to the OUs. As a result, in order to minimize the overall runtime weight programming overhead, we design a search strategy to find a near-optimal weight-to-OU mapping in the “Weight Allocator”

module, which takes both the weight data information from the trained DNN model, as well as the ReRAM PIM structure. Details of the “Weight Allocator” module will be presented in Section 5.1. As shown in Figure 2, in the OU-based PIM architecture, the entire weight is partitioned into OU-sized weight blocks. Meanwhile, the input feature maps for each DNN layer need to be properly arranged and converted to input voltages of the corresponding crossbar array. Existing work on OU-based PIM usually maps adjacent weight blocks to neighbouring OUs, so that the activation of OUs and corresponding input feature maps are straightforward [24, 36, 42]. On the other hand, in the above-mentioned mapping table generated by the proposed “Weight Allocator” module, we allow a weight block to be mapped to arbitrary physical OU in order to reduce the weight programming cost at runtime. To ensure the correctness of DNN inference, we design an “OU scheduler” module in the offline phase of the proposed architecture. The inputs of the “OU scheduler” are the DNN model which defines the computation between the input feature maps and the weight blocks, the PIM structure with crossbar and OU organization, as well as the generated “mapping table” which contains the weigh-to-OU mapping in each inference iteration. Based on this information, the “OU scheduler” module will generate a scheduling table to guide the activation order of OUs and corresponding input feature maps at runtime.

#### 4.2 Online Phase

In the proposed framework, the DNN inference process is divided into two iterative sub-stages. The “Weight Programming” stage prepares partial weights for the subsequent inference stage, subject to the capacity of PIM device. It writes OU-sized weight blocks to the designated physical OUs in accordance with the mapping table generated in the offline phase. Note that on a resource-constrained PIM device where the entire weights cannot be programmed onto the PIM at one go before DDN inference, the online weight programming cost has to be counted in the end-to-end inference latency.

The “inference” stage of the online phase starts once the required weight blocks for the next DNN inference iteration have been programmed onto the OUs. It reads the offline generated scheduling table to activate the computations on OUs with corresponding input feature map data. Similarly as in [36] and [42], we utilize the on-chip buffers in each PE for temporarily storing input and output feature maps, and accumulating the partial sum of the corresponding filters according to the type of computation. The “inference” stage and “weight programming” stage are iteratively executed till the DNN inference completes for each given input data.

### 5 STATIC SCHEDULING OF WEIGHT PROGRAMMING

In this section, we will introduce the design details of how to generate the weight-to-OU mapping table and the OU scheduling policies.

#### 5.1 Weight Allocator

• *Programming Delay Calculation Model.* In this work, we separate the DNN model weights into several weight regions and a weight region should be programmed to the ReRAM crossbars in one iteration. To minimize the programming latency during two iterations, we first build a model to formalize the process. The latency formula of a weight region programming latency is as follows:

$$latency = \max \left[ \sum_{OU_j \in CB_i}^{OU} (\alpha_{OU_j} \cdot m_{OU_j}^{CB_i} + \beta_{OU_j} \cdot n_{OU_j}^{CB_i}) | \forall CB_i \in CB \right]$$

We use  $CB$  to represent the number of crossbar arrays.  $OU_j$  represents an OU at a specific position (according to this OU’s distance from voltage drivers) in a crossbar, and  $OU$  represents the

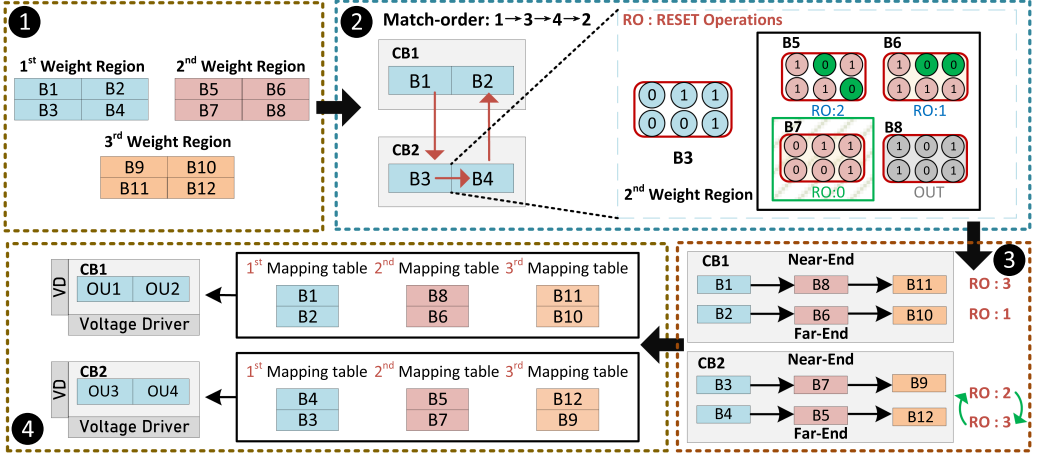


Fig. 5. Workflow of mapping table generation.

number of OUs (or weight blocks) that a crossbar array can hold. We use  $m_{OU_j}^{CB_i}$  to represent the number of rows that require RESET operations in the  $OU_j$  of  $CB_i$ , and  $n_{OU_j}^{CB_i}$  represents the number of rows that require SET operations in the  $OU_j$  of  $CB_i$ .  $\alpha_{OU_j}$  in the equation represents the average latency of a row with RESET operation in  $OU_j$ , and  $\beta_{OU_j}$  represents the average latency of a row with SET operation in  $OU_j$ .

As the formula shows, the programming latency of a single OU depends on the number of SET and RESET operations and its location. The latency of the RESET operation in the OU varies with the distance between the OU and the voltage drivers. The programming latency of the whole crossbar is the sum of the programming latencies of all its OUs. Finally, the programming latency of the whole weight region depends on the slowest crossbar.

When programming weights to ReRAM OUs, two unique intra-crossbar factors of ReRAM need to be considered: (1) The RESET operations at row-wise of ReRAM cells have more impact on the programming latency than SET operation; (2) The distance between the OU and the voltage drivers has a negative effect on the programming latency—the farther the OU is away from voltage drivers, the longer the programming latency it has. One inter-crossbar factor of ReRAM should also be considered: different ReRAM crossbars are programmed in parallel, so the programming latency for each weight region is dominated by the longest crossbar programming latency. To minimize the number of RESET operations, we first propose a similarity-based matching strategy. Its principle is that when one iteration ends, we will compared the to be programmed weight region with the values of current OUs. The weights are separated into weight blocks according to the size of OU (each OU stores one weight pattern). The similarity-based matching strategy will try to map these weight blocks to the corresponding OUs with least RESET operations. To balance the program latency of different crossbars, we further propose a crossbar balanced mapping strategy. At last, we take the OU location information into consideration and propose the a distance-aware mapping strategy.

• **Similarity-Based Matching Strategy.** We use the example in Figure 5 to illustrate the working process of the proposed mapping strategies. The example contains two logical crossbars (i.e., CB1 and CB2), every logical crossbar have two OUs (e.g., OU1 and OU2 in CB1). There are three weight regions. For each weight region, the neural network weights in each layer are evenly distributed to all the crossbars, and each iteration will maintain the number of OUs each crossbar can store for every model layer (determined by the number of weight blocks in a layer and the

**ALGORITHM 1:** Working Flow to Generate Mapping Table**Input:** ReRAM-based PIM Structure, CNN Model (weights);**Output:** Mapping Table for each crossbar array  $T$ ;

```

1:  $CB \leftarrow$  the number of crossbar arrays
2:  $OU \leftarrow$  the number of OU on each crossbar
3:  $WR \leftarrow$  the weight region being matched
4:  $RO \leftarrow$  the number of reset operations
5:  $Left[a][i] \leftarrow$  The number of OUs that layer where Block 'a' resides can still place on the
   crossbar 'i'
6:  $List[i][n] \leftarrow$  List of the 'n'th OU of the 'i'th crossbar
7: for  $n = 1$  to  $OU$  do
8:   for  $i = 1$  to  $CB$  do
9:     if  $n \% 2 == 0$  then
10:       $i = CB - i + 1$ 
11:    end if
12:    Get the 'n'th block on the 'i'th crossbar  $Block[i][n]$ 
13:     $min\_RO = Block[i][n].size / 8$ 
14:     $min\_Block = Block[0]$ 
15:    for  $k = 1$  to  $WR\_NEW.size$  do
16:      if  $Left[Block[i][n]][i] == 0$  then
17:        Continue
18:      else
19:        Get the 'k'th weight blocks of  $WR$   $Block[k]$ 
20:        Calculate  $RO$  from  $Block[i][n]$  to  $Block[k]$   $R$ 
21:        if  $R < min\_RO$  then
22:           $min\_RO = R$ 
23:           $min\_Block = Block[k]$ 
24:        end if
25:      end if
26:    end for
27:    Add  $min\_Block$  to  $List[i][n]$ 
28:    Remove  $min\_Block$  from  $WR$ ;
29:     $Left[Block[i][n]][i] -= 1$ 
30:  end for
31: end for
32: Count  $RO$  required for each  $List[i][n]$ 
33: Move the  $List[i][n]$  with minimum count to  $Far\_end$  of crossbar 'i' and so on
34: Assign Blocks in  $List$  to each mapping table  $T$ 

```

number of crossbars, i.e.,  $Left$ ), as shown in Algorithm 1, line 5. Suppose the first weight region has been temporarily programmed onto the OUs of ReRAM crossbars, when determining the positions of weight blocks in the second weight region, we propose a heuristic method to map weight blocks based on similarity to minimize each crossbar programming latency. We match each weight block with its most similar original weight block (in the first weight region that have already programmed onto OUs) that contains the minimum the number of rows with RESET operations. As shown in 2, when deciding which weight block in weight region 2 is mapped to B3, we calculated the number of RESET operations required for each unselected weight block in

the 2nd weight region. Note that RESET operations for certain bits can be performed in parallel for ReRAM. In this example, we set the paralleled number of bits to RESET as 3. In other words, if there are any cells in a row of one OU needs to be reprogrammed from 1 to 0 in the next iteration, the number of RESET operations should be added by one.

In practice, to achieve crossbar balance (make all the crossbars have similar programming latency), we make the weight region to be programmed as the search space, and use the weight blocks on OUs to search their most similar successors. The whole matching process is performed in a greedy approach. As shown in the figure, first, B1 searches the weight block with the highest similarity from weight region 2, and B8 is selected. In the following searches, B8 is no longer considered (make B8 in ❷ gray and attached behind B1 in the list of ❸). If a whole logical crossbar has found enough weight blocks in a certain layer (i.e., *Left* equals to zero), all weight blocks of this layer in weight region 2 will be kicked out of the search space like B8, as shown in Algorithm 1, line 16-17. With the process going, we found that the reset operation required for B7 programming to B3 is equal to 0, so B7 is stored after B2 in the list of ❹. After matching all the weight regions in this way, every weight block of the first weight region will form a programming list as ❺.

- **Balanced Mapping Strategy.** The above search process is a greedy process, so the search space of the OU that is searched first is the largest. As the progress being performed, the search space would become smaller and smaller, making it difficult to match the ideal weight blocks for the following OUs. If the matching process is carried out in serial in the way of OU first and then crossbar, the last crossbar may suffer from huge programming latency. So, we cyclically execute similarity-based matching strategy between all crossbars. As shown in ❷, to balance the search space of each crossbar, after B1 in crossbar1 is matched, we will select B3 in crossbar2 for the matching, then B4, B2, and so on. Through this matching sequence, the programming latency differences between the crossbars are greatly mitigated.

- **Distance-Aware Mapping Strategy.** Considering the variances of programming latencies of OUs in different locations in a crossbar, the OU with the farthest distance (i.e., denoted by the farthest OU) from the voltage drivers should prioritize the mapping procedure. The working process that combines the distance-aware mapping strategy and the similarity-based mapping strategy is as follows: When the search is completed for all weight regions of the DNN model, a programming list for each logical OU will be generated, as shown in Figure 5 (the length of each list depends on how many times the neural network needs to be programmed). We calculate the sum of the number of RESET operations required in each list, as shown in ❸, and adjust the position of each list inside the logical crossbar. The list with the smallest RESET operations would be placed at the farthest OU in the logical crossbar, and the list with the largest RESET operations is placed at the position of the nearest OU, and so on, as shown in ❹. With this approach, we can get the mapping table for each weight region. The mapping tables of these three weight regions are shown in Figure 5 ❺. They record the mapping of weight blocks to OUs in each crossbar.

## 5.2 OU Scheduler

This subsection elaborates the static OU scheduler to support the out-of-order OU activation. In the original design, the OUs in the crossbars are activated in sequence to perform the inference process. With the weight mapping strategy which aims to minimize the programming latency, we actually have broken up the mapping sequence between the input feature map and OUs (i.e., weight blocks) on the crossbars.

In order to activate OUs in the correct computing order, a static OU scheduler is proposed to capture the information of the assigned OU\_id (i.e., the relative location of an OU within a crossbar array) of weight blocks, and the corresponding input feature map block ID (IFM\_id). For each entry in

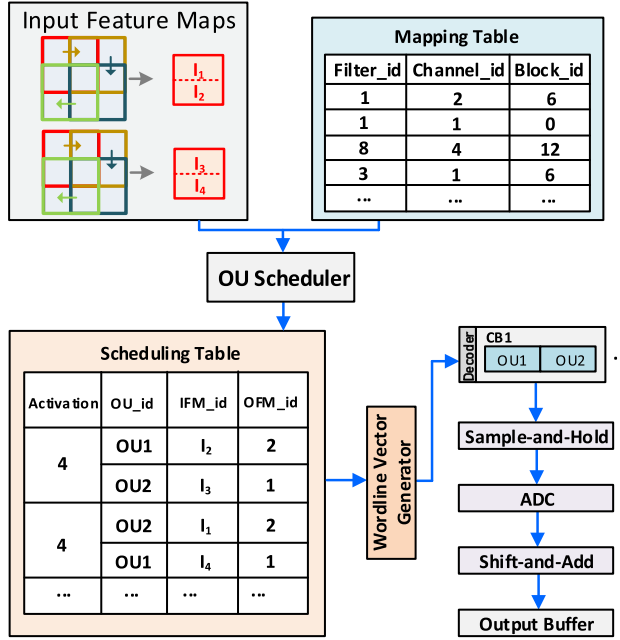


Fig. 6. Workflow of the offline OU scheduler.

the mapping table, the scheduler would first fetch the 3-tuple location information of weight blocks in the neural network consisting of  $\langle \text{Filter\_id}, \text{Channel\_id}, \text{Block\_id} \rangle$ . With the computation correspondence between the input feature map data and weight blocks in the DNN model, the number of activation, input index and output feature map index can be computed according to the location information of weight blocks. Thus, we can generate the OU scheduling table with the number of activations, OU ID and its corresponding input data and output feature map index as shown in Figure 6. The OU scheduling table records the activation information for each OU. Each entry in this table has four attributes: (1) the number of times a particular OU will be activated; (2) the OU id; (3) the corresponding input feature map id; (4) the corresponding output feature map id.

Each OU will be activated several times (in accordance with Activation) in a row based on IFM\_id. IFM\_id records the starting address of the input feature map data to be fed in for this particular weight block. Outputs with the same OFM\_id will be added together and stored at the designated position in the output address space for each DNN layer. Hence, we can sequentially feed input data fetched by IFM\_id from the first entry to the last one to the wordline vector generator, which produces the wordline activation vector at each cycle. After decoding the activation vector by the row decoder, the matching OU will be activated to execute analog MVM computations at runtime, as shown in Figure 6.

## 6 EVALUATION

### 6.1 Experimental Setup

We modify the MNSIM2.0 [33], a behavior-based PIM simulation platform, to evaluate the performance of the proposed ReRAM-based DNN accelerators. Table 2 shows the hardware configuration for each DNN process engine in our simulation. Based on the latest technology, scaling of ReRAM PIM devices [14, 35, 37], we set the area-constrained ReRAM PIM capacity to 16 Mb and 32 Mb for a small AI edge devices [7]. The OU-based crossbar working process adopts the pipeline design of

Table 2. Hardware Configuration

| <b>PE configuration</b> (500 MHz, 32 nm process, 8 PEs per tile) |                                                                                                       |
|------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| <b>Component</b>                                                 | <b>Spec</b>                                                                                           |
| Pooling unit / PE                                                | number: 1                                                                                             |
| Non-linear unit / PE                                             | number: 2                                                                                             |
| S&H / PE                                                         | number: $8 \times 16$                                                                                 |
| S&A / PE                                                         | number: 4                                                                                             |
| ADC / PE                                                         | number: 8; resolution: 5 bits<br>frequency: 1GSps                                                     |
| Memristor Array / PE                                             | number: 8; size: $512 \times 512$ ;<br>bits-per-cell: 1;<br>OU size: $16 \times 8$ and $16 \times 16$ |
| <b>HSPICE configuration</b>                                      |                                                                                                       |
| RESET                                                            | 3V, $90 \mu\text{A}$ per bit;<br>latency: $10 \sim 2 \times 10^3 \text{ns}$                           |
| SET                                                              | 3V, $98.6 \mu\text{A}$ per bit;<br>latency: 10ns                                                      |

SRE [36]. Since a 5-bit ADC is sufficient for a  $16 \times 16$  OU, we follow the equation given in [20] to scale the ADC cycle time. For the programming latency, especially the RESET latency variation at different positions in a crossbar, we simulate the rewrite performance of ReRAM using HSPICE. The simulation parameters are adopted from the state-of-the-art article that studies the voltage drop effects and its impact on ReRAM crossbar array RESET latency [45].

In the literature work, the ReRAM crossbar array size typically ranges from  $128 \times 128$  to  $1,024 \times 1,024$  [3, 6, 8, 9, 13, 21, 31, 36]. In general, a smaller crossbar size leads to significantly larger peripheral circuit area cost (e.g., ADC and DAC), and higher degree of processing parallelism. In this work, which focuses on area-constrained ReRAM PIM design, we choose  $512 \times 512$  crossbar array size to achieve the balance between performance and area cost. We set the OU size to  $16 \times 8$  and  $16 \times 16$ , two common sizes [36, 42, 44], to show the impact of OU size on performance. For the comparison, four different configurations are evaluated:

- (C1) 16 Mb ReRAM with  $16 \times 8$  OU size;
- (C2) 32 Mb ReRAM with  $16 \times 8$  OU size;
- (C3) 16 Mb ReRAM with  $16 \times 16$  OU size;
- (C4) 32 Mb ReRAM with  $16 \times 16$  OU size.

Note that increasing (decreasing) the number of crossbar arrays will improve (impair) the parallelism for both the “weight programming” and “inference” stages of the online phase, therefore has little impact on the relative ratio between the weight programming latency and inference latency.

We choose 10 DNN models as the benchmarks for the evaluations, including 5 image classification models and 5 NLP models. The image classification models include AlexNet, GoogleNet, ResNet50, ResNet101, and VGG16. For each network, we use both ImageNet and CIFAR-10 as the input dataset. NLP models include Transformer [27], BERT<sub>base</sub> [5], Chinese BERT<sub>base</sub>, ERNIE [43], and RoBERTa [15]. Compared with the image classification tasks, the NLP tasks contain a large portion of full connection (FC) layers, which can be converted into dot product operation and executed on ReRAM. Meanwhile, FC layers result in heavier weight programming cost w.r.t. the computation cost, compared with the convolution layers widely used in the image classification

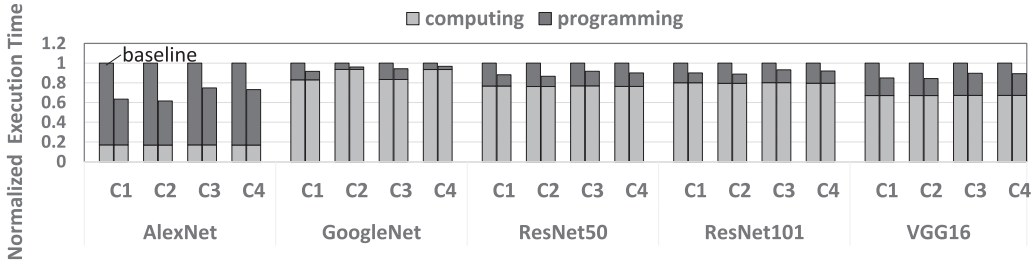


Fig. 7. Execution time breakdown of ImageNet inference tasks.

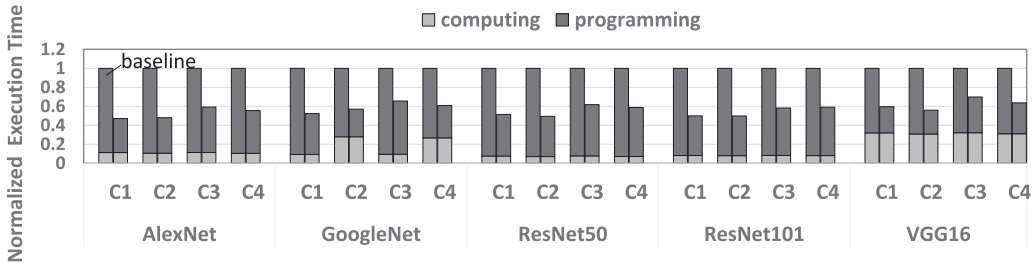


Fig. 8. Execution time breakdown of CIFAR-10 inference tasks.

models. Therefore, the weight programming overhead (w.r.t. the inference time) expects to be a more critical issue for NLP tasks executed on resource-constrained PIM devices. In our experiments, the weights and input of all models are quantized to 8 bits. We follow the mapping method of ISSAC [21], the positive and negative synapses weights mapping to the same crossbar, the weights are converted to 2's complements for CNN inference and weight programming.

In order to show the effectiveness of the proposed framework in terms of weight programming cost reduction, we compare the overall inference latency (including both weight programming and normal DNN inference time) with an intuitive “baseline” approach. In the “baseline” approach, we use the OU-based ReRAM accelerator without any offline weight programming cost optimization technique. Instead, weight blocks from the same layer are evenly distributed to all crossbar arrays following the same sequential OU index order. In the OU-based PIM architecture, the crossbar arrays execute in parallel, while only one OU in each crossbar can be activated in each cycle. Therefore, evenly distribution of weight blocks to crossbars in the “baseline” approach ensures maximized parallelism of computation. While for weight blocks assigned to the same crossbar, the “baseline” approach does not consider the programming cost, by sequentially mapping weight blocks to OUs.

## 6.2 Programming Cost Reduction

In this section, we evaluate the effectiveness of the proposed framework for weight programming cost reduction. We evaluate the proposed framework with different DNN models, input dataset, and ReRAM PIM device configurations. In the evaluation results, the total execution time of the online inference phase consists of the “programming” latency for the weight programming stage, and the “computing” latency for the DNN inference stage.

Figure 7 and Figure 8 present the performance breakdown of computing and programming latency for different image classification DNN inference tasks pre-trained with CIFAR-10 and ImageNet, respectively; while Figure 9 presents the results for the NLP models. Each neural network is

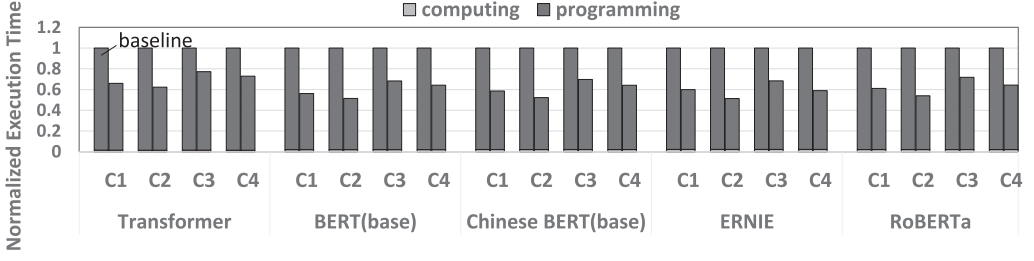


Fig. 9. Execution time breakdown of NLP inference tasks.

evaluated under 4 PIM device configurations as described in Section 6.1. All results are normalized to the overall execution time of the “baseline” approach in the corresponding setups.

**6.2.1 Impact of DNN Models.** The overall execution time reduction for ImageNet, CIFAR-10, and NLP are 11.99%, 40.29%, and 37.62%, respectively. And it can be observed that the performance gain are obtained for the “programming” latency reduction. On the other hand, the proposed framework has little impact on the “computing” latency of the normal DNN inference on ReRAM PIM.

In general, the proposed framework leads to the better overall performance gain on CIFAR-10 and NLP tasks, compared with results for ImageNet tasks. It is due to that different DNN models and input data sets have distinct weight-to-MAC (multiply-accumulate) ratios. As we mentioned in Section 3, the weight-to-MAC ratio is related to the structure of neural network (e.g., number of FC layers vs. convolution layers) and the size of convolution kernels. For example, the computing latency of ImageNet tasks accounts for an average of 69.09% of the overall execution time. Therefore, the proposed framework focuses on weight programming cost reduction has a smaller impact on their overall execution time. On the other hand, when the CIFAR-10 dataset is used as the input for the same set of DDN networks, the weight size remains similar, while the MAC count reduces significantly due to the smaller input images. For example, the weight-to-MAC ratio of VGG16 with ImageNet is 138.36 Mb/15.5 GMAC; while for CIFAR-10, the ratio is only 138.36 M/0.44 GMAC. Meanwhile, the NLP tasks are dominated by FC layers with less computation. As a result, the programming cost reduction achieves a higher contribution to the overall execution time for CIFAR-10 and NLP tasks.

We have further studied the results obtained for CIFAR-10 and NLP tasks. Even though NLP accounts for a higher proportion of programming latency compared with CIFAR-10, CIFAR-10 achieves better average weight programming cost reduction in the proposed framework. The reason is that the DNN models trained for CIFAR-10 have higher weight sparsity compared with that of the NLP models (e.g., 27% for AlexNet, 35% for ResNet50, and 29% for ResNet101). The high weight sparsity increases the probability of finding OU with high similarity, and reduces the weight programming cost.

**6.2.2 Impact of PIM Architecture.** Another observation we have made from the experimental results is that the ReRAM capacity has relatively small impact on both the computing latency and the programming cost for most of the DNN models and input data sets. For a given typical crossbar size, the number of crossbars in ReRAM increases linearly with the ReRAM capacity. Both the programming stage and inference stage benefit from the improved parallelism with more crossbar arrays. Therefore, only small differences in the execution time breakdown of computing and programming latency can be observed for different ReRAM capacities with the same OU size (i.e., between C1 and C2, or C3 and C4). However, the breakdown of computation latency and programming latency in GoogleNet is obviously affected by the ReRAM capacity. Because the number of

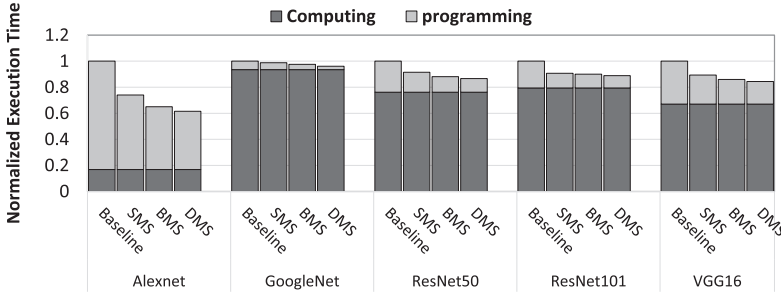


Fig. 10. Impact of Weight allocation strategies on ImageNet tasks.

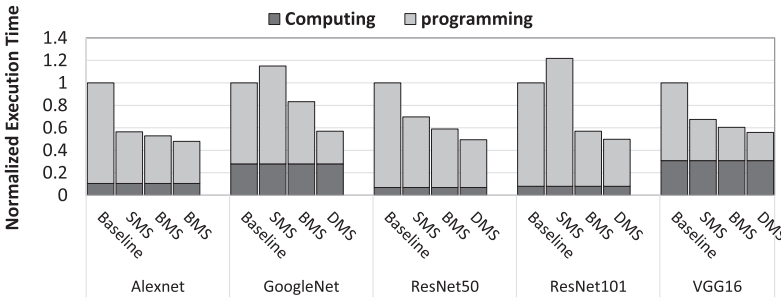


Fig. 11. Impact of Weight allocation strategies on CIFAR-10 tasks.

weight blocks in the second weight domain of GoogleNet is not enough to cover the whole ReRAM in 32Mb ReRAM, in the benchmark method, this part of weight blocks can be deployed only near the voltage driver, so the percentage of programming latency decreases significantly. With a 16-Mb ReRAM, the network still requires several iterations of programming and computation. In addition, in the 32-Mb ReRAM, the weight blocks in weight domain 2 have more matching space and their programming latency is optimized more significantly.

The experimental results also suggest better programming latency optimization effects for smaller OU size. For example, in Figure 9, for OU size of  $16 \times 8$  (configurations C1 and C2), the overall performance improvement is between 34.01% and 48.66% compared with the baseline approach. While for OU size of  $16 \times 16$ , the corresponding improvement is between 22.77% and 41.51%. The main reason is that for a small size OU, there is a better possibility of finding a weight block with high similarity (thus, low programming latency).

**6.2.3 Impact of Weight Allocation Strategies.** In Section 5.1, we define three sub-strategies in the proposed weight allocator module. We evaluate the effectiveness of individual sub-strategies for the weight programming cost reduction under a particular ReRAM PIM configuration C2. The results are shown in Figure 10, Figure 11, and Figure 12. The “SMS” sub-strategy adopts the similarity-based weight-to-OU match to minimize programming latency for individual OUs; “BMS” additionally exploits the round-robin search to balance the programming latency between all crossbar arrays. Finally, on top of “SMS” and “BMS”, “DMS” further considers the RESET latency variation due to the distance between a target OU and the voltage driver of the crossbar. In order to showcase how BMS affects the programming latency of the crossbar, we evaluated the programming latency of each crossbar for the first iteration of the cifar-10-based VGG16 under a particular ReRAM PIM configuration C2. The result is shown in Figure 13.

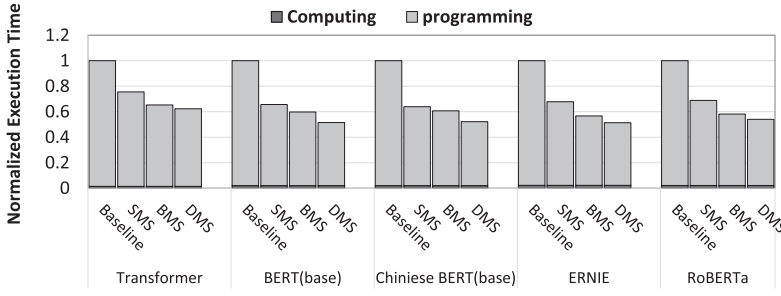


Fig. 12. Impact of Weight allocation strategies on NLP tasks.

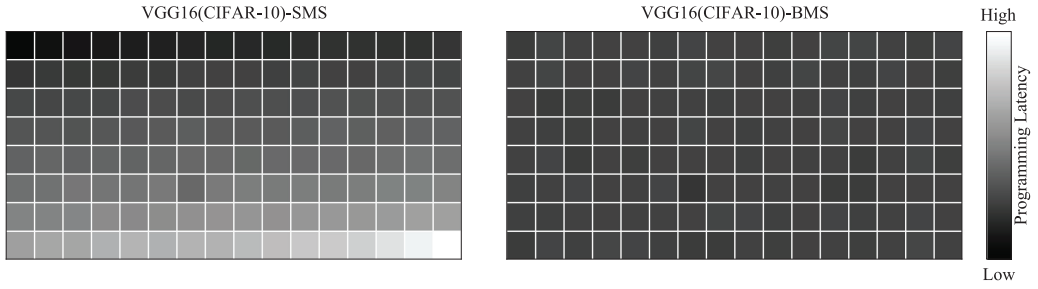


Fig. 13. BMS Impact on Crossbar Programming Latency.

If we apply only “SMS” weight-to-OU mapping search strategy, the performance improvement is 18.88% on average. In several cases, e.g., GoogleNet(CIFAR-10) and ResNet101(CIFAR-10) in Figure 11, the weight programming cost even increases w.r.t. the baseline approach without similarity matching. It is because the sequential intra-crossbar weight-to-OU similarity search leads to unbalanced programming latency cost among different crossbars. The OUs in the crossbars searched at last have few opportunities to find a low-latency matching of weight blocks. Meanwhile, the programming latency of the whole ReRAM depends on the slowest crossbar. As a result, “SMS” with sequential search induces significantly higher programming latency for the last searched crossbars, as shown in subfigure VGG16 (CIFAR-10)-SMS of Figure 13. Note that the “baseline” randomly assigns weight blocks to OUs and balances the number of weight blocks on each crossbar for a given DNN layer. Therefore, it is possible that imbalance between programming latency cost on different crossbars are small in some cases.

Built on top of “SMS”, the “BMS” search strategy overcomes the above-mentioned problem with a cyclic search among all crossbars, as shown in subfigure VGG16 (CIFAR-10)-BMS of Figure 13, which leads to a 30% performance improvement w.r.t. the baseline. Finally, “DMS” combines both “SMS” and “BMS”, as well as a fine-grained programming latency model of ReRAM crossbars, which leads to a performance improvement of 34.77% in average over baseline.

### 6.3 Integration with Weight Pruning Techniques

In this section, we show that the proposed weight programming cost reduction framework can be seamlessly integrated with existing weight pruning techniques. In principle, weight pruning techniques reduce the weight size, which benefits both the weight programming cost and the inference computation latency. Therefore, as long as the pruned weight size is still larger than the available ReRAM PIM capacity (which is true for all the DNN models in our evaluation), online weight programming is required for the DNN inference.

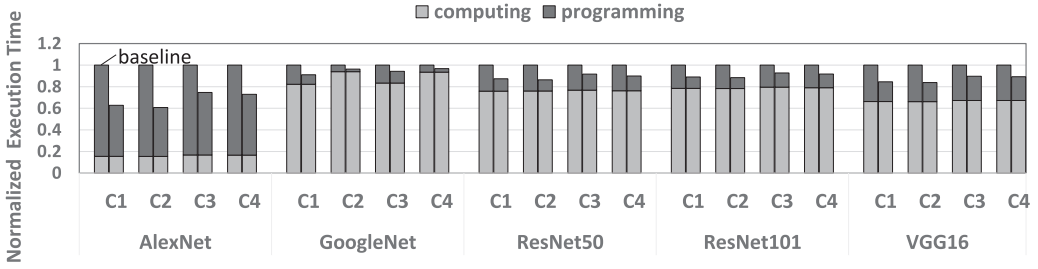


Fig. 14. Execution time breakdown of ImageNet inference task with weight pruning.

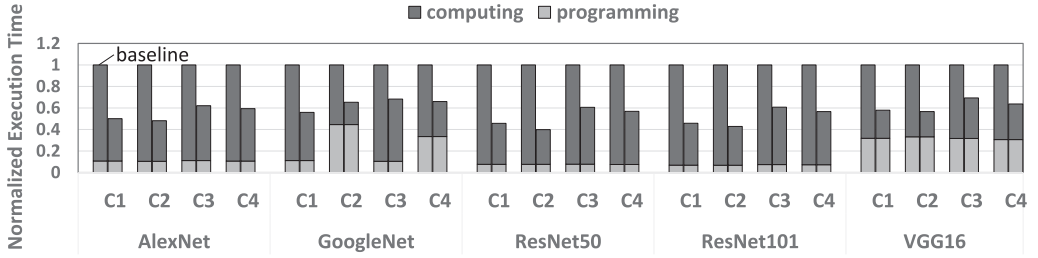


Fig. 15. Execution time breakdown of CIFAR-10 inference task with weight pruning.

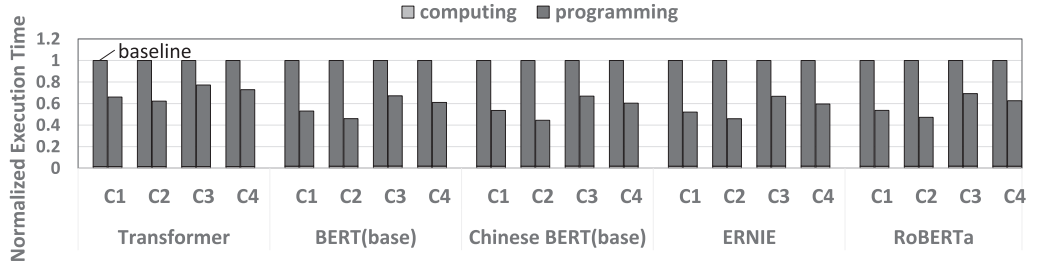


Fig. 16. Execution time breakdown of NLP inference task with weight pruning.

We combine the proposed framework with the weight pruning technique in SRE [36]. SRE utilizes the sparsity of the weights to design customized pruning schemes for the OU-based ReRAM structure. In particular, the ORC strategy performs column compression on the weights distributed to the OU, by pruning the rows with all 0 values in the OU. Since ORC aims to reduce unnecessary computation during neural network inference, it is orthogonal to our approach to reducing the programming latency of weights. In order to combine the proposed framework with ORC, we first divide the weights of the quantized 8-bit neural network into OU-sized weight blocks. During this process, we prune the rows whose weights are all 0. And the pruned weights are scheduled to map onto the OUs in the inference stage by the method proposed in this article.

Figure 14, Figure 15, and Figure 16 show the inference time with the breakdown of computing and programming latency for ImageNet, CIFAR-10, and NLP tasks, respectively. The SRE-ORC weight pruning technique is applied to both the baseline approach and the proposed framework. The average performance improvement for all the DNN models with weight pruning is 32.75%, compared with an average improvement of 31.57% without any pruning technique (i.e., the average improvement in Figure 7, Figure 8, and Figure 9). The results show that the proposed framework is still effective when pruning techniques are applied to the DNN models.

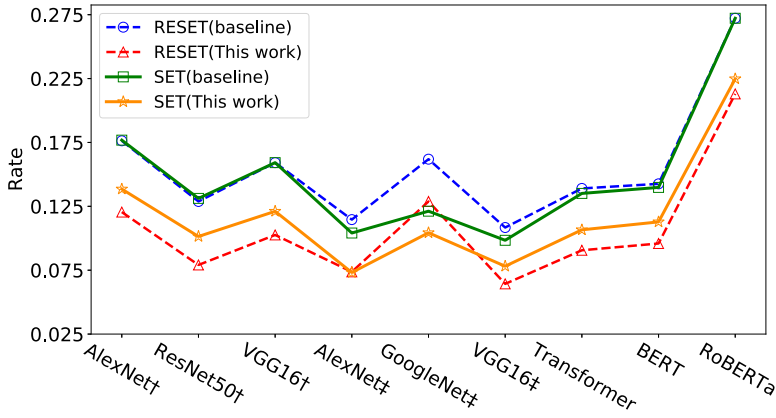


Fig. 17. Impact of the proposed framework on the programming rate of ReRAM cells. † Neural Network trained base on ImageNet. ‡ Neural Network trained base on CIFAR10.

It should be noted that the ORC strategy not only reduces unnecessary calculations in the computing stage, but also reduces the number of weights that need to be programmed into ReRAM. Therefore, the application of ORC strategy will not have a distinct impact on the performance breakdown of computing and programming. In addition, in the experimental configuration, the input and weight are quantized to 8 bits, and the number of columns of OU is a multiple of 8. Therefore, only part of the decimal value 0 is cut by ORC strategy, which will not reduce the accuracy of neural network. See Figure 17.

#### 6.4 Endurance

ReRAM has demonstrated superior endurance ( $>10^{12}$  [7]). Taking AlexNet as an example, inference of an input image with 32Mb ReRAM PIM requires 15 iterative weight programming stages at runtime. Assume a frame rate of 30 input images per second, in the worst case (every cell needs to be programmed in each iteration), the ReRAM is expected to work continuously for 70 years. Note that the weight programming cost is induced by the deployment of a relatively large DNN inference model on a resource-constrained PIM device.

In practice, our proposed method significantly reduces the number of cells that needs to be programmed in each iteration, which benefits both programming cost as well as the ReRAM endurance. In this section, some neural networks are selected based on a specific ReRAM configuration C2, and the impact of the proposed framework on the programming rate of ReRAM cells is evaluated, and the results are shown in Figures 5-10. It should be emphasized that the number of unit operations counted in this experiment is not the number of operations programmed in parallel by multiple units as described in the previous section, but the count is performed on a single unit basis. In the baseline approach for the ImageNet, CIFAR-10, and NLP tasks participating in the experiment, to complete all iterations of a neural network inference, an average of 31.05%, 23.62%, and 36.70% of the units require SET or RESET operations. It is important to note that each unit will only be executed with either SET or RESET operation, and both operations will not be executed at the same time. After applying the weight similarity-based DNN inference scheduling framework proposed in this article to these neural networks, the ratio of units requiring programming operations to complete all iterations of a neural network inference decreases to 22.11%, 17.41%, and 28.12% for the three neural networks, reducing the need for programming operations by 8.94%, 6.21%, and 8.58% of the total number of units in the ReRAM chip, respectively.

## 6.5 Overhead

The proposed framework for weight programming cost reduction brings additional timing and space overhead during the online inference phase, in order to manifest the mapping table and scheduling table as shown in Figure 4. According to the evaluation result, the timing overhead to activate OUs according to the offline constructed scheduling table is negligible. Note that the timing overhead for scheduling of OU activation and manipulate of the partial results across different crossbar arrays also presents in other OU-based PIM architecture (e.g., [36] and [42]). For the storage overhead, the additional buffer space required by the mapping table is 0.22 MB to 1.28 MB, while the scheduling table is 0.24 MB to 1.32 MB for the four configurations in our evaluation, depending on the weight size of DNN models and ReRAM organization. The storage overhead is substantially smaller than the corresponding DNN model's original weight data (refer to Table 1), and can be stored in the off-chip DRAM memory.

## 7 CONCLUSION

In this article, we identify the challenge of online weight programming cost for DNN inference on a resource-constrained ReRAM PIM device. A framework consisting of an offline phase to generate weight-to-OU mapping and scheduling information, and an online phase to iteratively proceed the weight programming and normal inference. The experimental results show that, the proposed mapping strategies achieve effective weight programming overhead reduction and overall performance improvement for various DNN models.

## ACKNOWLEDGMENTS

We sincerely thank the anonymous reviewers for their insightful and valuable feedback.

## REFERENCES

- [1] Aayush Ankit, Izzat El Hajj, Sai Rahul Chalamalasetti, Geoffrey Ndu, Martin Foltin, R. Stanley Williams, Paolo Faraboschi, Wen-mei W. Hwu, John Paul Strachan, Kaushik Roy, et al. 2019. PUMA: A programmable ultra-efficient memristor-based accelerator for machine learning inference. In *Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems*. 715–731.
- [2] Wei-Hao Chen, Kai-Xiang Li, Wei-Yu Lin, Kuo-Hsiang Hsu, Pin-Yi Li, Cheng-Han Yang, Cheng-Xin Xue, En-Yu Yang, Yen-Kai Chen, Yun-Sheng Chang, et al. 2018. A 65nm 1Mb nonvolatile computing-in-memory ReRAM macro with sub-16ns multiply-and-accumulate for binary DNN AI edge processors. In *ISSCC'18*.
- [3] Ping Chi, Shuangchen Li, Cong Xu, Tao Zhang, Jishen Zhao, Yongpan Liu, Yu Wang, and Yuan Xie. 2016. Prime: A novel processing-in-memory architecture for neural network computation in reram-based main memory. In *ISCA'16* (2016).
- [4] Chaoqun Chu, Yanzhi Wang, Yilong Zhao, Xiaolong Ma, Shaokai Ye, Yunyan Hong, Xiaoyao Liang, Yinhe Han, and Li Jiang. 2020. PIM-Prune: Fine-grain DCNN pruning for crossbar-based process-in-memory architecture. In *DAC'20*.
- [5] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. BERT: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805* (2018).
- [6] Saransh Gupta, Mohsen Imani, Behnam Khaleghi, Venkatesh Kumar, and Tajana Rosing. 2019. RAPID: A ReRAM processing in-memory architecture for DNA sequence alignment. In *2019 IEEE/ACM International Symposium on Low Power Electronics and Design (ISLPED)*. IEEE, 1–6.
- [7] Chung-Wei Hsu, I-Ting Wang, Chun-Li Lo, Ming-Chung Chiang, Wen-Yueh Jang, Chen-Hsi Lin, and Tuo-Hung Hou. 2013. Self-rectifying bipolar TaO x/TiO 2 RRAM with superior endurance over 10-12 cycles for 3D high-density storage-class memory. In *2013 Symposium on VLSI Technology*. IEEE, T166–T167.
- [8] Yu Ji, Youyang Zhang, Xinfeng Xie, Shuangchen Li, Peiqi Wang, Xing Hu, Youhui Zhang, and Yuan Xie. 2019. FPSA: A full system stack solution for reconfigurable reram-based nn accelerator architecture. In *24th International Conference on Architectural Support for Programming Languages and Operating Systems*. 733–747.
- [9] Hai Jin, Cong Liu, Haikun Liu, Ruikun Luo, Jiahong Xu, Fubing Mao, and Xiaofei Liao. 2021. ReHy: A ReRAM-based digital/analog hybrid PIM architecture for accelerating CNN training. *IEEE Transactions on Parallel and Distributed Systems* (2021).

- [10] Wantong Li, Xiaoyu Sun, Shanshi Huang, Hongwu Jiang, and Shimeng Yu. 2022. A 40-nm MLC-RRAM compute-in-memory macro with sparsity control, on-chip write-verify, and temperature-independent ADC references. *IEEE Journal of Solid-State Circuits* (2022).
- [11] Weitao Li, Pengfei Xu, Yang Zhao, Haitong Li, Yuan Xie, and Yingyan Lin. 2020. Timely: Pushing data movements and interfaces in pim accelerators towards local and in time domain. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 832–845.
- [12] Meng-Yao Lin, Hsiang-Yun Cheng, Wei-Ting Lin, Tzu-Hsien Yang, I-Ching Tseng, Chia-Lin Yang, Han-Wen Hu, Hung-Sheng Chang, Hsiang-Pang Li, and Meng-Fan Chang. 2018. DL-RSIM: A simulation framework to enable reliable ReRAM-based accelerators for deep learning. In *2018 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. IEEE, 1–8.
- [13] Fangxin Liu, Wenbo Zhao, Zhezhi He, Zongwu Wang, Yilong Zhao, Yongbiao Chen, and Li Jiang. 2021. Bit-transformer: Transforming bit-level sparsity into higher performance in ReRam-based accelerator. In *2021 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. IEEE, 1–9.
- [14] Qi Liu, Bin Gao, Peng Yao, Dong Wu, Junren Chen, Yachuan Pang, Wenqiang Zhang, Yan Liao, Cheng-Xin Xue, Wei-Hao Chen, et al. 2020. A fully integrated analog ReRAM based 78.4 TOPS/W compute-in-memory chip with fully parallel MAC computing. In *ISSCC'20*.
- [15] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. ROBERTA: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692* (2019).
- [16] Anni Lu, Xiaochen Peng, Yandong Luo, and et al. 2021. A runtime reconfigurable design of compute-in-memory-based hardware accelerator for deep learning inference. *ACM Trans. Design Autom. Electr. Syst.* (2021).
- [17] Anni Lu, Xiaochen Peng, Yandong Luo, and Shimeng Yu. 2020. Benchmark of the compute-in-memory-based DNN accelerator with area constraint. *IEEE Trans. on Very Large Scale Integration (VLSI) Systems* 28, 9 (2020), 1945–1952.
- [18] Anni Lu, Xiaochen Peng, and Shimeng Yu. 2021. Compute-in-RRAM with limited on-chip resources. In *AICAS'21*.
- [19] Manqing Mao, Xiaochen Peng, Rui Liu, and et al. 2019. MAX 2: An ReRAM-based neural network accelerator that maximizes data reuse and area utilization. *J. Emerg. Sel. Topics Circuits Syst.* (2019).
- [20] Mehdi Saberi, Reza Lotfi, Khalil Mafinezhad, and Wouter A. Serdijn. 2011. Analysis of power consumption and linearity in capacitive digital-to-analog converters used in successive approximation ADCs. *IEEE Transactions on Circuits and Systems I: Regular Papers* 58, 8 (2011), 1736–1748.
- [21] Ali Shafiee, Anirban Nag, Naveen Muralimanohar, Rajeev Balasubramonian, John Paul Strachan, Miao Hu, R Stanley Williams, and Vivek Srikumar. 2016. ISAAC: A convolutional neural network accelerator with in-situ analog arithmetic in crossbars. *ISCA'16*.
- [22] Karen Simonyan and Andrew Zisserman. 2015. Very deep convolutional networks for large-scale image recognition. In *ICLR'15*.
- [23] Linghao Song, Xuehai Qian, Hai Li, and Yiran Chen. 2017. Pipelayer: A pipelined rram-based accelerator for deep learning. In *HPCA'17*.
- [24] Zhuoran Song, Dongyue Li, Zhezhi He, Xiaoyao Liang, and Li Jiang. 2021. ReRAM-sharing: Fine-grained weight sharing for ReRAM-based deep neural network accelerator. In *ISCAS'21*.
- [25] Chen-Yang Tsai, Chin-Fu Nien, Tz-Ching Yu, Hung-Yu Yeh, and Hsiang-Yun Cheng. 2021. RePIM: Joint exploitation of activation and weight repetitions for in-ReRAM DNN acceleration. In *DAC'21*.
- [26] Yen-Ting Tsou, Kuan-Hsun Chent, Chia-Lin Yang, Hsiang-Yun Cheng, Jian-Jia Chen, and Der-Yu Tsai. 2022. This is SPATEM! A spatial-temporal optimization framework for efficient inference on ReRAM-based CNN accelerator. In *2022 27th Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 702–707.
- [27] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in Neural Information Processing Systems* 30 (2017).
- [28] Chengning Wang, Dan Feng, Jingning Liu, Wei Tong, Bing Wu, and Yang Zhang. 2017. DAWs: Exploiting crossbar characteristics for improving write performance of high density resistive memory. In *ICCD'17*.
- [29] Chengning Wang, Dan Feng, Wei Tong, Jingning Liu, Bing Wu, Wei Zhao, Yang Zhang, and Yiran Chen. 2020. Improving write performance on cross-point RRAM arrays by leveraging multidimensional non-uniformity of cell effective voltage. *TC'20* (2020).
- [30] Wen Wen, Youtao Zhang, and Jun Yang. 2018. Wear leveling for crossbar resistive memory. In *2018 55th ACM/ESDA/IEEE Design Automation Conference (DAC)*. IEEE, 1–6.
- [31] Wen Wen, Youtao Zhang, and Jun Yang. 2019. ReNEW: Enhancing lifetime for ReRAM crossbar based neural network accelerators. In *2019 IEEE 37th International Conference on Computer Design (ICCD)*. IEEE, 487–496.
- [32] Wen Wen, Youtao Zhang, and Jun Yang. 2020. Accelerating 3D vertical resistive memories with opportunistic write latency reduction. In *2020 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. IEEE, 1–8.

- [33] Lixue Xia, Boxun Li, Tianqi Tang, and et al. 2017. MNSIM: Simulation platform for memristor-based neuromorphic computing system. *TCAD* 37, 5 (2017), 1009–1022.
- [34] Cong Xu, Dimin Niu, Naveen Muralimanohar, Rajeev Balasubramonian, Tao Zhang, Shimeng Yu, and Yuan Xie. 2015. Overcoming the challenges of crossbar resistive memory architectures. In *HPCA'15*.
- [35] Cheng-Xin Xue, Je-Min Hung, Hui-Yao Kao, and et al. 2021. A 22nm 4Mb 8b-precision ReRAM computing-in-memory macro with 11.91 to 195.7 TOPS/W for tiny AI edge devices. In *ISSCC'21*.
- [36] Tzu-Hsien Yang, Hsiang-Yun Cheng, Chia-Lin Yang, and et al. 2019. Sparse ReRAM engine: Joint exploration of activation and weight sparsity in compressed neural networks. In *ISCA'19*.
- [37] Jong-Hyeok Yoon, Muya Chang, Win-San Khwa, Yu-Der Chih, Meng-Fan Chang, and Arijit Raychowdhury. 2021. 29.1 A 40nm 64Kb 56.67 TOPS/W read-disturb-tolerant compute-in-memory/digital RRAM macro with active-feedback-based read and in-situ write verification. In *ISSCC'21*.
- [38] Geng Yuan, Payman Behnam, Zhengang Li, and et al. 2021. FORMS: Fine-grained polarized ReRAM-based in-situ computation for mixed-signal DNN accelerator. In *ISCA'21*.
- [39] Hang Zhang, Nong Xiao, Fang Liu, and Zhiguang Chen. 2016. LEADER: Accelerating ReRAM-based main memory by leveraging access latency discrepancy in crossbar arrays. In *2016 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 756–761.
- [40] Yang Zhang, Dan Feng, Jingning Liu, Wei Tong, Bing Wu, and Caihua Fang. 2017. A novel ReRAM-based main memory structure for optimizing access latency and reliability. In *54th Annual Design Automation Conference 2017*. 1–6.
- [41] Yang Zhang, Dan Feng, Wei Tong, Yu Hua, Jingning Liu, Zhipeng Tan, Chengning Wang, Bing Wu, Zheng Li, and Gaoxiang Xu. 2018. CACF: A novel circuit architecture co-optimization framework for improving performance, reliability and energy of ReRAM-based main memory system. *ACM Transactions on Architecture and Code Optimization (TACO)* 15, 2 (2018), 1–26.
- [42] Yuhao Zhang, Zhiping Jia, Yungang Pan, Hongchao Du, Zhaoyan Shen, Mengying Zhao, and Zili Shao. 2020. PATTPIM: A practical ReRAM-based DNN accelerator by reusing weight pattern repetitions. In *DAC'20*.
- [43] Zhengyan Zhang, Xu Han, Zhiyuan Liu, Xin Jiang, Maosong Sun, and Qun Liu. 2019. ERNIE: Enhanced language representation with informative entities. *arXiv preprint arXiv:1905.07129* (2019).
- [44] Yang-Lin Zheng, Wei-Yi Yang, Ya-Shu Chen, and Ding-Hung Han. 2022. An energy-efficient inference engine for a configurable ReRAM-based neural network accelerator. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* (2022).
- [45] Farzaneh Zokaee and Lei Jiang. 2020. Mitigating voltage drop in resistive memories by dynamic reset voltage regulation and partition reset. In *HPCA'20*.

Received 26 January 2023; revised 21 May 2023; accepted 22 July 2023